

4.2. Estilos y temas en Flutter



DESARROLLO DE APLICACIONES WEB MULTIPLATAFORMA NIVEL 3.

UNIDAD 4:

DISEÑO Y ESTILOS EN FLUTTER

Contenido

Introducción.....	4
Relación con diseños complejos y layouts adaptativos:	4
Importancia de los estilos y temas para la consistencia visual:.....	4
Importancia de utilizar estilos y temas para crear interfaces de usuario atractivas:	5
Diferencias entre estilos y temas:	5
Estilos en Flutter	6
Creación de estilos en Flutter	6
Propiedades disponibles en TextStyle	7
Aplicación de estilos a múltiples widgets:	8
Estrategias para aplicar estilos a múltiples widgets:	9
Beneficios de utilizar estas estrategias:.....	11
Estilos personalizados:.....	12
Creación de clases personalizadas para estilos:.....	12
Ventajas de los estilos personalizados:	14
Temas en Flutter	15
Widget Theme:.....	15
Estructura del Widget Theme:	16
Propiedades disponibles en Theme:.....	16
Aplicación de temas a la aplicación:	18
Uso del widget Theme en el widget raíz	18
Herencia de temas.....	19
Cambio dinámico de temas	20
Consideraciones generales:.....	21
Temas personalizados:	21
Importancia de los temas personalizados:	21

Creación de clases personalizadas para temas:	22
Ventajas de los temas personalizados:.....	23
Recursos adicionales.....	24

Introducción

En el mundo del desarrollo de interfaces de usuario, los estilos y temas son herramientas fundamentales para crear aplicaciones visualmente atractivas y consistentes.

- Estilos: Los estilos se enfocan en la apariencia individual de los widgets, como el color de la fuente, el tamaño del texto, los bordes, etc. Se aplican a widgets específicos para controlar su aspecto visual.
- Temas: Por otro lado, los temas definen la apariencia general de toda la aplicación, estableciendo una paleta de colores, tipografía, estilos predeterminados para diferentes tipos de widgets y otros aspectos visuales globales.

Relación con diseños complejos y layouts adaptativos:

Los estilos y temas son esenciales para crear diseños complejos y layouts adaptativos en Flutter.

- Estilos: Permiten reutilizar y combinar estilos fácilmente, creando una base sólida para diseños complejos. Además, facilitan la adaptación de estilos a diferentes tamaños de pantalla y densidades de píxeles.
- Temas: Proporcionan una base coherente para el diseño de toda la aplicación, asegurando que los elementos visuales se integren armoniosamente, incluso en layouts adaptativos que cambian según la orientación del dispositivo o el tamaño de la pantalla.

Importancia de los estilos y temas para la consistencia visual:

- Consistencia: Los estilos y temas garantizan que todos los elementos de la interfaz de usuario tengan una apariencia similar, creando una experiencia visual coherente y profesional para los usuarios.
- Reducción de errores: Al definir estilos y temas predefinidos, se minimizan los errores visuales y se asegura que la aplicación mantenga una estética uniforme en todas sus pantallas.
- Ahorro de tiempo: Reutilizar estilos y temas ahorra tiempo y esfuerzo al desarrollador, ya que no es necesario definir la apariencia de cada widget de forma individual.

Importancia de utilizar estilos y temas para crear interfaces de usuario atractivas:

- **Atracción visual:** Los estilos y temas permiten crear interfaces de usuario visualmente atractivas y agradables para los usuarios, lo que aumenta la satisfacción y el engagement con la aplicación.
- **Jerarquía visual:** Mediante el uso adecuado de estilos y temas, se puede establecer una jerarquía visual clara en la interfaz, guiando la atención del usuario hacia los elementos más importantes.
- **Personalización:** Los estilos y temas pueden personalizarse para adaptarse a la marca o identidad visual de la aplicación, creando una experiencia única y memorable para los usuarios.

Diferencias entre estilos y temas:

Característica	Estilos	Temas
Alcance	Aplicado a widgets individuales	Aplicado a toda la aplicación
Control	Define la apariencia de un widget específico	Define la apariencia general de la aplicación
Herencia	Se pueden heredar entre widgets	Se aplican a todos los widgets de la aplicación por defecto
Prioridad	Los estilos locales tienen prioridad sobre los estilos heredados del tema	Los temas tienen mayor prioridad que los estilos locales

Estilos en Flutter



Los estilos son la clave para crear interfaces de usuario consistentes y atractivas en Flutter

Creación de estilos en Flutter

La clase `TextStyle` de Flutter te permite definir y aplicar estilos a tus widgets de texto con gran precisión.

El siguiente fragmento de código Dart define un objeto de tipo `TextStyle`, el cual representa un conjunto de propiedades para el estilo de un texto en Flutter

Dart

```
TextStyle(  
  color: Colors.blue, // Color del texto  
  fontSize: 16.0, // Tamaño de la fuente  
  fontWeight: FontWeight.bold, // Peso de la fuente  
  fontFamily: 'Roboto', // Tipo de fuente  
),
```

Propiedades disponibles en TextStyle

La clase TextStyle ofrece una amplia gama de propiedades para controlar diversos aspectos de la apariencia del texto. A continuación, se presenta un resumen de las propiedades más comunes y útiles:

Propiedades básicas:

- color: Define el color del texto. Se puede usar un objeto Color o un valor hexadecimal (Colors.blue).
- fontSize: Establece el tamaño de la fuente en unidades de píxeles.
- fontWeight: Controla el grosor de la fuente (FontWeight.normal, FontWeight.bold, etc.).
- fontStyle: Indica el estilo de la fuente (FontStyle.normal, FontStyle.italic).
- fontFamily: Especifica el tipo de fuente a usar (Roboto, Arial, etc.).

Propiedades de decoración:

- decoration: Agrega decoración al texto (subrayado, tachado).
- decorationColor: Define el color de la decoración (línea, sombra).
- decorationThickness: Controla el grosor de la decoración.
- decorationStyle: Establece el estilo de la decoración (sólido, punteado).

Propiedades de espaciado:

- letterSpacing: Ajusta el espaciado entre letras en unidades de píxeles.
- wordSpacing: Controla el espaciado entre palabras en unidades de píxeles.
- height: Define la altura de la línea de texto.
- leadingScale: Ajusta el espaciado entre líneas en relación al tamaño de la fuente.

Propiedades de alineación:

- textAlign: Controla la alineación horizontal del texto (TextAlign.left, TextAlign.center, etc.).
- textBaseline: Define la alineación vertical del texto.

Propiedades de comportamiento:

- overflow: Determina el comportamiento cuando el texto no cabe en el espacio disponible (TextOverflow.ellipsis, TextOverflow.clip, etc.).
- maxLines: Limita el número máximo de líneas de texto que se muestran.

- `softWrap`: Permite que el texto se ajuste al ancho disponible.

Propiedades de sombra:

- `shadow`: Define una sombra para el texto.
- `shadowColor`: Establece el color de la sombra.
- `shadowOffset`: Controla el desplazamiento de la sombra.
- `shadowRadius`: Ajusta el radio de desenfoque de la sombra.

Propiedades de transformación:

- `textTransform`: Transforma el texto a mayúsculas, minúsculas o según otras reglas (`TextTransform.uppercase`, `TextTransform.lowercase`, etc.).
- `locale`: Define el idioma del texto.

Propiedades avanzadas:

- `inherit`: Controla si el estilo se hereda por widgets descendientes.
- `useTextShade`: Habilita el uso de sombra de texto para mejorar la legibilidad.
- `fontFamilyFallback`: Especifica una fuente alternativa si la principal no está disponible.
- `textDirection`: Define la dirección de lectura del texto (`TextDirection.ltr`, `TextDirection.rtl`).

Aplicación de estilos a múltiples widgets:

Aplicar estilos a múltiples widgets es fundamental en el desarrollo de interfaces de usuario con Flutter por varias razones:

- **Consistencia**: Garantiza una apariencia uniforme en toda la aplicación, creando una experiencia visual coherente para los usuarios.
- **Reutilización**: Permite ahorrar tiempo y código al evitar repetir la definición de estilos para cada widget individual.
- **Organización**: Mejora la legibilidad y mantenibilidad del código, facilitando la comprensión y modificación posterior.
- **Flexibilidad**: Proporciona mayor flexibilidad para adaptar el estilo de los widgets según las necesidades específicas de cada caso.

Estrategias para aplicar estilos a múltiples widgets:

Flutter ofrece diversas estrategias para aplicar estilos a múltiples widgets de manera eficiente:

Uso de variables para almacenar estilos:

Utilizar variables para almacenar estilos en Flutter implica empaquetar un conjunto de especificaciones de estilo para poder aplicarlas de manera conjunta. Esto significa que defines un estilo una vez y luego lo reutilizas en diferentes partes de tu aplicación mediante una variable.

El proceso implica:

- Definir variables que almacenan objetos TextStyle con los estilos deseados.
- Utilizar estas variables al aplicar estilos a diferentes widgets.

Ejemplo:

El siguiente fragmento de código crea un estilo de texto personalizado llamado titulo y lo aplica a dos widgets de texto diferentes, lo que les da la misma apariencia (tamaño de fuente negrita y color morado)

Dart

```
final TextStyle titulo = TextStyle(  
  fontSize: 24.0,  
  fontWeight: FontWeight.bold,  
  color: Colors.purple,  
);
```

```
Text('Título de la sección', style: titulo),
```

```
Text('Otro título', style: titulo),
```

Usa el código con precaución.

Herencia de estilos:

La herencia de estilos en Flutter te permite empaquetar un conjunto de especificaciones de estilo en una clase base y luego permitir que las clases derivadas hereden esas especificaciones. Esto significa que defines un estilo base una vez y luego lo puedes reutilizar en diferentes widgets sin necesidad de repetir el código.

El proceso implica:

1. Crea un widget personalizado que defina un estilo base.
2. Los widgets descendientes heredan el estilo base y pueden modificarlo si es necesario.

Veamos un ejemplo:

El siguiente código define un widget `MiWidget` que muestra dos textos. El primer texto utiliza el estilo base definido en la clase. El segundo texto hereda el estilo base y lo modifica para tener un peso de fuente en negrita utilizando `copyWith`.

Dart

```
class MiWidget extends StatelessWidget {  
  
  final TextStyle estiloBase = TextStyle(  
  
    color: Colors.black,  
  
    fontSize: 16.0,  
  
  );  
  
  @override  
  
  Widget build(BuildContext context) {  
  
    return Column(  
  
      children: [  
  
        Text('Texto 1', style: estiloBase),  
  
      ],  
    );  
  }  
}
```

```
Text('Texto 2', style: estiloBase.copyWith(fontWeight:
FontWeight.bold)),
],
);
}
}
```

Combinación de estilos:

Utiliza el método `copyWith()` para combinar un estilo base con modificaciones específicas.

Permite crear variaciones de un estilo base sin necesidad de definir nuevos objetos `TextStyle`.

Ejemplo:

El siguiente fragmento de código Dart crea un widget de texto con un estilo personalizado que combina dos estilos diferentes.

```
Dart
Text(
  'Texto combinado',
  style: TextStyle(
    color: Colors.blue,
    fontSize: 18.0,
  ).copyWith(fontWeight: FontWeight.bold),
),
```

Usa el código con precaución.

Beneficios de utilizar estas estrategias:

- Reducción de Código: Se escribe menos código, mejorando la legibilidad y mantenibilidad.

- Mayor Eficiencia: Se ahorra tiempo al reutilizar estilos en diferentes partes de la aplicación.
- Flexibilidad: Se facilita la adaptación de estilos a diferentes contextos.
- Consistencia Visual: Se asegura una apariencia uniforme en toda la interfaz de usuario.

Estilos personalizados:

Los estilos personalizados son una herramienta fundamental en el desarrollo de interfaces de usuario con Flutter. Permiten:

- Consistencia: Asegurar una apariencia uniforme en toda la aplicación, creando una experiencia visual coherente para los usuarios.
- Reutilización: Ahorrar tiempo y código al evitar repetir la definición de estilos para cada widget individual.
- Organización: Mejorar la legibilidad y mantenibilidad del código, facilitando la comprensión y modificación posterior.
- Flexibilidad: Proporcionar mayor flexibilidad para adaptar el estilo de los widgets según las necesidades específicas de cada caso.

Creación de clases personalizadas para estilos:

Flutter ofrece dos enfoques principales para crear estilos personalizados:

Clases personalizadas:

Se define una clase que hereda de la clase `TextStyle` de Flutter.

Esta clase puede incluir propiedades adicionales para controlar aspectos específicos del estilo, como márgenes, bordes, sombras, etc.

Se pueden crear métodos para generar diferentes variaciones del estilo.

Veamos un ejemplo:

El siguiente código define una clase personalizada `MiEstiloTexto` que te permite crear estilos de texto con un margen horizontal adicional.

Dart

```
class MiEstiloTexto extends TextStyle {  
  
    final double margenHorizontal; // Propiedad adicional para  
    márgenes horizontales
```

```
MiEstiloTexto({
  required Color color,
  required double fontSize,
  FontWeight fontWeight = FontWeight.normal,
  this.margenHorizontal = 0.0,
}) : super(
  color: color,
  fontSize: fontSize,
  fontWeight: fontWeight,
);
}
```

Usa el código con precaución.

Extensión de la clase TextStyle:

Se utilizan extensiones de la clase TextStyle para agregar nuevos métodos o propiedades.

Estos métodos o propiedades pueden simplificar la creación de estilos personalizados.

Veamos un ejemplo:

El siguiente código permite agregar fácilmente un margen horizontal simétrico a un estilo de texto existente en Flutter utilizando una extensión.

Dart

```
extension TextStyleExtension on TextStyle {
  TextStyle withMargin(double horizontalMargin) {
    return copyWith(
      margin: EdgeInsets.symmetric(horizontal: horizontalMargin),
    );
  }
}
```

```
);  
}  
}
```

```
Text('Texto con margen', style: TextStyle(color: Colors.black, fontSize:  
16.0).withMargin(16.0)),
```

Usa el código con precaución.

Ventajas de los estilos personalizados:

- Mayor Control: Permiten un control más preciso sobre la apariencia de los widgets.
- Facilidad de Uso: Simplifican la creación y aplicación de estilos complejos.
- Flexibilidad: Se pueden adaptar fácilmente a diferentes necesidades de diseño.
- Mantenibilidad: Mejoran la organización y legibilidad del código.
- Reutilización: Promueven la reutilización de estilos en diferentes partes de la aplicación.

Temas en Flutter



Los temas son un componente fundamental en Flutter para definir la apariencia general de una aplicación. Permiten:

- Consistencia: Asegurar una apariencia uniforme en toda la aplicación, creando una experiencia visual coherente para los usuarios.
- Personalización: Adaptar la apariencia de la aplicación a diferentes preferencias, como modo claro u oscuro, o temas específicos de la marca.
- Heredabilidad: Aplicar estilos a varios widgets de forma jerárquica, simplificando la definición de estilos.
- Flexibilidad: Modificar estilos de forma centralizada sin afectar a cada widget individualmente.

Widget Theme:

El widget Theme es el elemento central para la gestión de temas en Flutter. Se encarga de:

- Proporcionar estilos a los widgets descendientes: Los widgets que se encuentran dentro del árbol de widgets del widget Theme heredarán los estilos definidos en él.
- Establecer un tema predeterminado: La aplicación puede tener un tema predeterminado que se aplica automáticamente a los widgets que no están dentro de un widget Theme específico.

- Permitir la personalización: Los desarrolladores pueden crear sus propios temas personalizados para adaptarlos a las necesidades específicas de la aplicación.

Por lo tanto, todo lo que tiene que ver con la creación y la gestión de temas pasa por este widget.

Estructura del Widget Theme:

Se compone de dos partes principales:

- data: Un objeto ThemeData que contiene las propiedades para definir los estilos del tema.
- child: El widget que heredará los estilos del tema.

Veamos un ejemplo:

El siguiente fragmento de código Dart crea un widget Theme que define un tema personalizado y lo aplica a un widget hijo.

```
Dart

Theme(
  data: ThemeData(
    // Propiedades para definir estilos
  ),
  child: WidgetHijo(), // Widget que heredará los estilos del tema
);
```

Propiedades disponibles en Theme:

Los temas en Flutter se definen mediante la aplicación de propiedades al widget Theme.

El widget Theme funciona como un contenedor que engloba a otros widgets y les transmite los estilos que se establecen en su propiedad data.

El widget ThemeData proporciona una amplia gama de propiedades para definir diferentes aspectos de la apariencia de la aplicación, incluyendo:

- Colores: Colores primarios, secundarios, de fondo, de texto, etc.
- Tipografía: Estilos de fuente para títulos, encabezados, párrafos, etc.
- Formas: Estilos para botones, entradas de texto, etc.
- Animaciones: Duraciones y curvas de animación para transiciones, etc.
- Barra de aplicaciones: Estilo de la barra de aplicaciones, color, icono, etc.
- Tema oscuro: Configuración específica para el modo oscuro.

Ejemplo de Definición de un Tema:

El siguiente código crea un tema personalizado con colores primarios y secundarios, y define estilos de texto para títulos y párrafos. Luego, aplica este tema a un widget Scaffold que representa la estructura básica de una aplicación Flutter con una barra de aplicaciones y un título. Como resultado, la aplicación tendrá una apariencia consistente con los estilos definidos en el tema.

```
Dart

Theme(
  data: ThemeData(
    primaryColor: Colors.blue, // Color primario
    accentColor: Colors.lightBlue, // Color secundario
    textTheme: TextTheme(
      headline1: TextStyle(fontSize: 24.0, fontWeight:
FontWeight.bold),
      body1: TextStyle(fontSize: 16.0),
    ),
  ),
  child: Scaffold(
    appBar: AppBar(
```

```

        title: Text('Mi aplicación'),
      ),
      body: Center(
        child: Text('Ejemplo de tema'),
      ),
    ),
  );

```

Aplicación de temas a la aplicación:

Como hemos empezado a ver en el epígrafe anterior, los temas son un componente fundamental en Flutter para definir la apariencia general de una aplicación. Permiten crear interfaces de usuario consistentes, atractivas y personalizables. En este apartado, te mostraré cómo aplicar temas en tu aplicación Flutter de forma efectiva.

Uso del widget Theme en el widget raíz

El primer paso para aplicar temas de manera general a una aplicación o a una web es envolver el widget raíz (generalmente MyApp) en un widget Theme. Esto asegurará que todos los widgets descendientes hereden los estilos definidos en el tema.

Veamos un ejemplo:

```

Widget build(BuildContext context) {
  return Theme(
    data: ThemeData(
      // Propiedades para definir estilos
    ),
    child: MyHomePage(), // Widget raíz de tu aplicación
  );
}

```

Este código crea un tema personalizado y lo aplica a la aplicación Flutter envolviendo el widget raíz (MyHomePage) en un widget Theme. Al hacerlo, garantiza que todos los widgets descendientes de MyHomePage heredarán los estilos definidos en el tema, creando una interfaz de usuario consistente y atractiva.

Herencia de temas

Los temas se heredan de forma jerárquica en Flutter. Esto significa que los widgets dentro del árbol de widgets del widget Theme heredarán los estilos definidos en él.

Ejemplo:

```
Dart

Theme(
  data: ThemeData(
    primaryColor: Colors.blue,
  ),
  child: Scaffold(
    appBar: AppBar(
      title: Text('Mi aplicación'),
    ),
    body: Center(
      child: Text('Ejemplo de tema'),
    ),
  ),
);
```

Usa el código con precaución.

En este ejemplo, el widget Text dentro del widget Center heredará el color primario azul definido en el tema.

Cambio dinámico de temas

Es posible cambiar el tema de forma dinámica durante la ejecución de la aplicación. Para ello, puedes utilizar el Provider de Flutter o un paquete de gestión de estado como bloc o riverpod.

Ejemplo con Provider:

Dart

```
class ThemeProvider extends ChangeNotifier {  
  ThemeData _themeData = ThemeData(primaryColor:  
Colors.blue);
```

```
  ThemeData get themeData => _themeData;
```

```
  void setThemeData(ThemeData themeData) {  
    _themeData = themeData;  
    notifyListeners();  
  }  
}
```

```
// En el widget raíz
```

```
Widget build(BuildContext context) {  
  final themeProvider = Provider.of<ThemeProvider>(context);  
  
  return Theme(  
    data: themeProvider.themeData,  
    child: MyHomePage(),
```

```
);  
}
```

```
// En otro widget donde se cambia el tema
```

```
Provider.of<ThemeProvider>(context).setThemeData(ThemeData(primaryColor: Colors.red));
```

Este código implementa un sistema para cambiar dinámicamente el tema de una aplicación Flutter. El `ThemeProvider` actúa como un proveedor centralizado que almacena el tema actual y notifica a los widgets que escuchan los cambios. Al utilizar `Provider`, puedes cambiar el tema desde cualquier lugar de la aplicación y los widgets que dependen del tema se actualizarán automáticamente.

Consideraciones generales:

- Los temas son una herramienta poderosa para crear interfaces de usuario atractivas y consistentes.
- Úsalos de manera responsable para evitar complejidad innecesaria en tu código.
- Considera la accesibilidad al elegir los colores y estilos de tu tema.
- Explora diferentes paquetes de gestión de temas para mayor flexibilidad.

Temas personalizados:

Los temas personalizados son una herramienta fundamental en Flutter para crear interfaces de usuario consistentes, atractivas y adaptables a las necesidades específicas de tu aplicación. Te permiten controlar la apariencia de los elementos de la interfaz, como colores, tipografía, botones, barras de aplicaciones y más.

Importancia de los temas personalizados:

- **Consistencia:** Aseguran que todos los elementos de la interfaz tengan una apariencia uniforme, creando una experiencia visual agradable y coherente.

- **Facilidad de mantenimiento:** Permiten modificar la apariencia general de la aplicación desde un solo lugar, facilitando los cambios y actualizaciones futuras.
- **Personalización:** Brindan la flexibilidad para crear interfaces únicas que reflejen la identidad de tu marca o proyecto.
- **Accesibilidad:** Permiten ajustar los colores y estilos para garantizar una experiencia accesible para todos los usuarios, incluyendo aquellos con daltonismo o dificultades visuales.
- **Reutilización:** Puedes usar el mismo tema en diferentes proyectos o pantallas de la aplicación, ahorrando tiempo y esfuerzo.

Creación de clases personalizadas para temas:

El proceso de creación es el siguiente:

1. Define una clase que extienda ThemeData:

Dart

```
class MyThemeData extends ThemeData {  
  
    // Propiedades para personalizar estilos  
  
}
```

Es importante definir esta clase como base para definir los estilos personalizados del tema.

2. Personaliza las propiedades del tema:
 - a. Define propiedades como `primaryColor`, `accentColor`, `textTheme`, `appBarTheme`, `scaffoldBackgroundColor`, etc.
 - b. Establece los valores para cada propiedad según tus necesidades.

Es necesario establecer valores para cada propiedad de acuerdo a las necesidades del proyecto

3. Utiliza la clase personalizada en el widget Theme:

Dart

```
Theme(  
  
    data: MyThemeData(),
```

```
child: MyApp(),  
);
```

4. Extensión del widget Theme:

- a. Puedes extender el widget Theme para crear tu propio widget con propiedades personalizadas.
- b. Esto te permite encapsular aún más la lógica del tema y reutilizarlo en diferentes partes de la aplicación.

Ventajas de los temas personalizados:

- Control total sobre la apariencia: Define la apariencia de cada elemento de la interfaz con precisión.
- Flexibilidad para adaptar a tu marca: Crea un tema que refleje la identidad visual de tu marca o proyecto.
- Facilita la implementación de accesibilidad: Ajusta los colores y estilos para garantizar una experiencia accesible para todos los usuarios.
- Simplifica el mantenimiento del código: Modifica la apariencia general desde un solo lugar, reduciendo el código repetitivo.
- Promueve la reutilización de código: Utiliza el mismo tema en diferentes proyectos o pantallas de la aplicación.

Recursos adicionales

General

- Documentación oficial de Flutter sobre estilos y temas: <https://docs.flutter.dev/ui/widgets/styling>
- Artículo sobre estilos y temas en Flutter: <https://medium.com/flutter-community/themes-in-flutter-part-1-75f52f2334ea>

Estilos

- Tutorial sobre estilos en Flutter: <https://www.freecodecamp.org/news/tag/flutter/>
- Documentación oficial de Flutter sobre estilos de texto: <https://api.flutter.dev/flutter/dart-ui/TextStyle-class.html>

Temas

- Documentación oficial de Flutter sobre temas: <https://docs.flutter.dev/get-started/flutter-for/web-devs>
- Artículo sobre temas en Flutter: <https://medium.com/flutter-community/themes-in-flutter-part-1-75f52f2334ea>
- Tutorial sobre creación de temas personalizados en Flutter: <https://www.youtube.com/watch?v=HoeXeculaU8>

¡Enhorabuena por completar esta unidad didáctica sobre estilos y temas en Flutter!

Espero que esta unidad didáctica te haya sido útil e informativa. Te animo a seguir explorando el mundo de Flutter y a crear aplicaciones hermosas y funcionales.

¡Mucha suerte en tu camino como desarrollador Flutter!

¡Hasta la próxima!